

Esta página ha sido traducida por una máquina.

# Más información sobre la arquitectura de microservicios

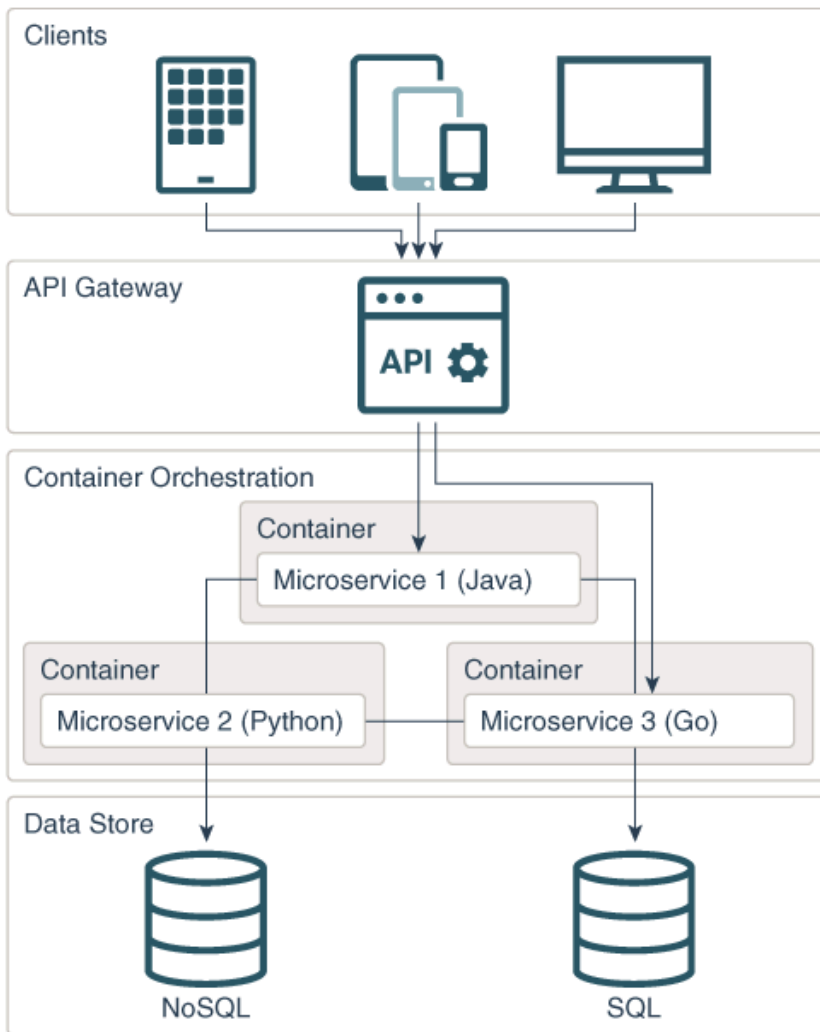
Las aplicaciones modernas se crean creando algunos servicios que se crean de forma independiente. Esto proporciona agilidad y velocidad de comercialización para solucionar problemas e introducir nuevas funciones.

Este paradigma se conoce como arquitectura de microservicios, aunque el número de servicios que se unen para una aplicación completa puede estar en los cientos (microservicios) o en las decenas (macro-servicios). Hay nuevos términos que también se utilizan para los monolitos modulares llamados modulitos, y Spring Modulith es uno de esos ejemplos.

Aunque muchas organizaciones ya tienen una arquitectura de microservicios, los despliegues de microservicios están asociados con la complejidad, y los despliegues exitosos aún están en curso en la mayoría de las organizaciones. Este manual de soluciones intenta simplificar la creación y el despliegue de microservicios modernos con la popular plataforma Spring Boot junto con la infraestructura en la nube, los contenedores, Kubernetes y la plataforma de backend como servicio con Oracle Database.

Si desea diseñar una aplicación que sea multilingüe, fácilmente escalable, fácil de mantener y desplegar, de alta disponibilidad y que minimice los fallos, utilice la arquitectura de microservicios para diseñar y desplegar una aplicación en la nube. En una arquitectura de microservicios, cada microservicio posee una tarea sencilla y se comunica con los clientes o con otros microservicios mediante mecanismos de comunicación ligeros, como solicitudes de API de REST.

En el siguiente diagrama se muestra la arquitectura de una aplicación que consta de varios microservicios.



[Descripción de la ilustración microservice architecture.png](#)

Los microservicios le permiten diseñar su aplicación como una recopilación de servicios acoplados de forma flexible. Los microservicios siguen el modelo de no compartir nada y se ejecutan como procesos sin estado. Este enfoque permite ampliar y mantener la aplicación con mayor facilidad.

- La capa API es el punto de entrada para todas las solicitudes de cliente a un microservicio. La capa API también permite que los microservicios se comuniquen entre sí a través de HTTP, gRPC y TCP/UDP.
- La capa lógica se centra en una única tarea de negocio, minimizando las dependencias de los otros microservicios. Esta capa se puede escribir en un lenguaje diferente para cada microservicio.
- La capa del almacén de datos proporciona un mecanismo de persistencia, como un motor de almacenamiento de base de datos, archivos log, etc. Considere el uso de un almacén de datos persistente independiente para cada microservicio. Oracle ofrece contenedores de base de datos con varias bases de datos conectables que facilitan que los microservicios aíslen los datos para garantizar la seguridad, la alta disponibilidad y la ampliación. Además, las aplicaciones SaaS pueden aprovechar el

multi-arrendamiento de forma segura. Además, una base de datos convergente ofrece la variedad de datos bajo un mismo techo para obtener información más detallada de los datos.

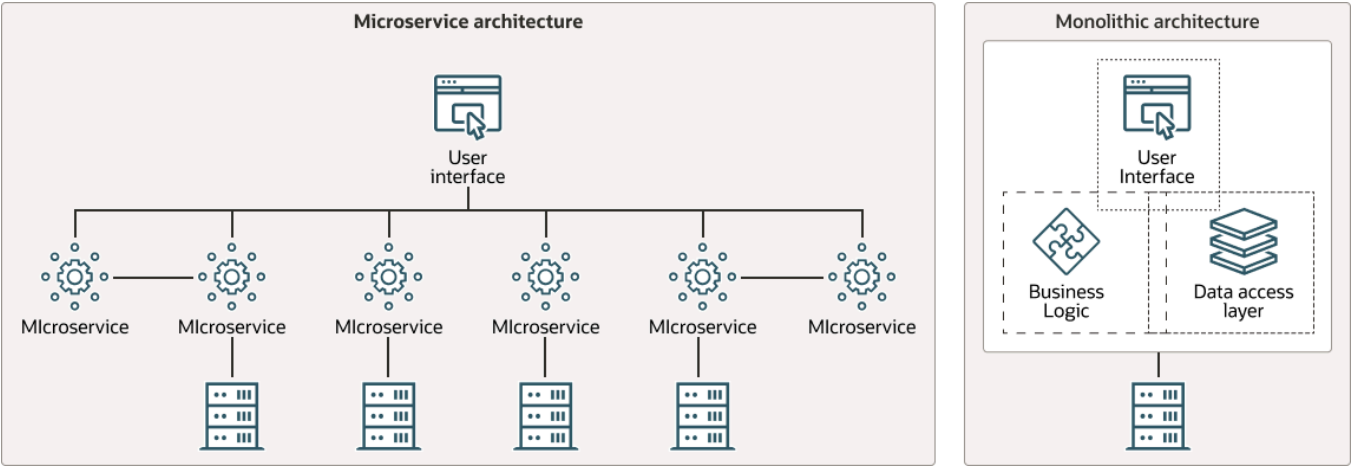
Normalmente, cada microservicio se ejecuta en un *contenedor* que proporciona un entorno de tiempo de ejecución ligero.

## Diferencias entre microservicios y arquitecturas monolíticas

Antes de empezar a diseñar aplicaciones con la arquitectura de microservicios, debe comprender cómo esta arquitectura difiere de la arquitectura monolítica tradicional.

El diseño de aplicaciones se centra en resolver requisitos de negocio e implantar la lógica de negocio. En una arquitectura monolítica, toda la aplicación se crea como una sola unidad que contiene toda la lógica de negocio. En la arquitectura de microservicios, la lógica de negocio se organiza como varios servicios débilmente acoplados.

En la siguiente ilustración, se muestran las arquitecturas monolíticas y de microservicios.



[Descripción de la ilustración monolithic vs microservice.png](#)

En la siguiente tabla se resumen las diferencias entre los microservicios y las arquitecturas monolíticas.

| Característica                 | Arquitectura de microservicios                                                                                  | Arquitectura monolítica                                                              |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Diseño de la Unidad            | La aplicación consta de servicios acoplados de forma flexible. Cada servicio admite una única tarea de negocio. | Toda la aplicación está diseñada, desarrollada y desplegada como una sola unidad.    |
| Reutilización de funcionalidad | Los microservicios definen las API que exponen su funcionalidad a cualquier cliente. Los clientes               | La oportunidad de reutilizar la funcionalidad en todas las aplicaciones es limitada. |

| Característica                       | Arquitectura de microservicios                                                                                                                                                                               | Arquitectura monolítica                                                                                                                                                                        |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                      | podrían incluso ser otras aplicaciones.                                                                                                                                                                      |                                                                                                                                                                                                |
| Comunicación dentro de la aplicación | Para comunicarse entre sí, los microservicios de una aplicación utilizan el modelo de comunicación solicitud-respuesta. La implantación típica utiliza llamadas de API de REST basadas en el protocolo HTTP. | Los procedimientos internos (llamadas de funciones) facilitan la comunicación entre los componentes de la aplicación. No es necesario limitar el número de llamadas a procedimientos internos. |
| Flexibilidad tecnológica             | Cada microservicio se puede desarrollar utilizando un lenguaje de programación y un marco que mejor se adapte al problema que el microservicio está diseñado para resolver.                                  | Normalmente, toda la aplicación se escribe en un único lenguaje de programación.                                                                                                               |
| Gestión de Datos                     | Descentralizado: cada microservicio puede utilizar su propia base de datos.                                                                                                                                  | Centralizado: toda la aplicación utiliza una o más bases de datos.                                                                                                                             |
| Despliegue                           | Cada microservicio se despliega de forma independiente, sin que esto afecte a los demás microservicios de la aplicación.                                                                                     | Cualquier cambio, aunque pequeño, requiere un nuevo despliegue y reinicio de toda la aplicación.                                                                                               |
| Capacidad de mantenimiento           | Los microservicios son simples, centrados e independientes. La aplicación es más fácil de mantener.                                                                                                          | A medida que aumenta el ámbito de aplicación, el mantenimiento del código se vuelve más complejo.                                                                                              |
| Resiliencia                          | La funcionalidad de la aplicación se distribuye entre varios servicios. Si un microservicio falla, la funcionalidad que ofrecen los otros microservicios sigue estando disponible.                           | Un fallo en cualquier componente puede afectar a la disponibilidad de toda la aplicación.                                                                                                      |
| Posibilidades de ampliación          | Cada microservicio se puede escalar de forma independiente al resto de servicios.                                                                                                                            | Se debe escalar toda la aplicación, incluso cuando el requisito de negocio es escalar solo determinadas partes de la aplicación.                                                               |

## Consideraciones para la adopción de la arquitectura de microservicios

Al diseñar una nueva aplicación, tenga en cuenta la arquitectura de microservicios para aplicaciones que requieren altos niveles de escalabilidad, flexibilidad y fiabilidad. Puede utilizar un lenguaje de programación y un marco diferentes para desarrollar cada componente.

Considere la posibilidad de migrar una aplicación monolítica a microservicios si puede dividir la funcionalidad de la aplicación en servicios específicos, cada uno con un ámbito limitado. Para aplicaciones monolíticas complejas que no se pueden migrar a la arquitectura de microservicios, considere desarrollar solo la nueva funcionalidad como microservicios.

Las interdependencias entre servicios pueden afectar a la cantidad de aplicaciones que no están disponibles durante un nuevo despliegue de servicio. Resuelva estos problemas de dependencia durante el diseño de la aplicación.

Refactorizar una aplicación monolítica es difícil. Si desea utilizar la arquitectura de microservicios, planifíquela desde el inicio del proyecto.

Tenga en cuenta las complejidades de los sistemas distribuidos al desarrollar microservicios y recuerde que las llamadas remotas son lentas y pueden fallar. En función del caso de uso, debe equilibrar la consistencia, la disponibilidad y la tolerancia a particiones.

Esté preparado para la complejidad operativa que implica la arquitectura de microservicios. Recomendamos cumplir los siguientes requisitos previos antes de mover una aplicación monolítica a la arquitectura de microservicios:

- **Aprovisionamiento rápido:** capacidad para aprovisionar servidores rápidamente
- **Supervisión básica:** capacidad para detectar problemas de servicio y problemas empresariales. Los problemas de servicio incluyen disponibilidad del servicio, errores funcionales y problemas de rendimiento
- **Despliegue rápido de aplicaciones:** capacidad de desplegar cualquier servicio rápidamente en los entornos de prueba y producción

Asegúrese de que las ventajas de la arquitectura de microservicios superen el costo.

## **Acerca del mecanismo de comunicación en una arquitectura de microservicios**

En una arquitectura de microservicios, los servicios se ejecutan en varios servidores. La comunicación entre estos servicios se produce a través de protocolos como HTTP, AMQP y TCP. Los protocolos más utilizados son los de mensajería HTTP/REST y asíncrona.

Una API de REST para servicios web suele utilizar el protocolo HTTP. Con los métodos HTTP (como GET, POST, PUT y DELETE), los clientes pueden acceder y manipular los recursos de la aplicación mediante un localizador de recursos (URL) uniforme.

Los clientes envían una solicitud a un servicio a través de una API de REST que representa un punto de entrada a la funcionalidad de la aplicación. Los clientes pueden comunicarse con microservicios directamente o a través de un gateway de API.

Un patrón de gateway de API define un único punto de entrada para todas las solicitudes a los servicios. Cuando llega una solicitud de cliente, el gateway de API enruta la solicitud al servicio adecuado.

Una variante del patrón de gateway de API es el patrón de *backend para frontend*, que define un gateway de API independiente para cada tipo de cliente (por ejemplo, un gateway para clientes móviles y otro para aplicaciones web).

Minimizar la comunicación entre los servicios es una práctica recomendada. Cuando la comunicación es esencial, es preferible la comunicación asíncrona. El servicio que envía la solicitud puede seguir funcionando sin esperar una respuesta.

Las colas de mensajes y los sistemas de transmisión son métodos ideales para proporcionar comunicación asíncrona y, cuando se integran en la base de datos, proporcionan semántica transaccional para una operación de datos y el envío de un mensaje. Esto hace que los microservicios sean más sencillos y escalables de desplegar. El uso de API de REST hace que la comunicación entre microservicios sea síncrona y, por lo general, limita la ampliación.

## Acerca de los servicios y los roles necesarios

Esta solución requiere los siguientes servicios:

- Oracle Cloud Infrastructure Database (una variedad de opciones, incluida Oracle Autonomous Database)
- OCI Kubernetes Engine
- Oracle Backend for Spring Boot and Microservices

Estos son los roles necesarios para cada servicio.

| Nombre de servicio: Rol                                                        | Necesario para...                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Oracle Cloud Infrastructure Database: <code>SYSTEM</code> o <code>ADMIN</code> | Cree un usuario para proxy de acceso a la base de datos al configurar Oracle Backend for Spring Boot and Microservices. Aunque <code>SYSTEM</code> o <code>ADMIN</code> funcionarán, tienen demasiados privilegios y no se deben utilizar en entornos de producción. |
| OCI Kubernetes Engine: <code>CLUSTER_MANAGE</code>                             | Cree un cluster de OCI Kubernetes Engine y un pool de nodos con tres nodos de trabajador.                                                                                                                                                                            |
| Oracle Backend for Spring Boot and Microservices: <code>ROLE_ADMIN</code>      | Instalar Oracle Backend for Spring Boot and Microservices.                                                                                                                                                                                                           |

Consulte [Productos, soluciones y servicios de Oracle](#) para obtener lo que necesita.

